

1. (A) Below is the output of running a fully fledged vanilla implementation of Misra Gries on S1.txt:

label	"a"	"b"	"p"	"d"	"s"	"f"	"c"	"v"	"k"	"t"	"n"
count	772,865	472,863	2	1	1	1	233,850	2	1	1	1
ratio	0.25762	0.15762	0.0	0.0	0.0	0.0	0.07795	0.0	0.0	0.0	0.0

Table 1. Output of Misra Gries algorithm on stream S1. See [Listing 1](#) for a fuller explanation on how these frequencies have been generated.

Below is the output of running a fully fledged vanilla implementation of Misra Gries on S2.txt:

label	"t"	"b"	"a"	"h"	"j"	"s"	"c"	"u"	"f"	"i"	"o"
count	0	710,525	1,911,225	0	0	0	311,750	0	0	0	0
ratio	0.0	0.17763	0.47781	0.0	0.0	0.0	0.07794	0.0	0.0	0.0	0.0

Table 2. Output of Misra Gries algorithm on stream S2. See [Listing 1](#) for fuller explanation on how these frequencies have been generated.

— Elements that “might” occur at least 25% of the time

We note that for a fix entry, say q , in the stream, the following inequality holds between the ground-truth frequency f_q and the estimated frequency $\hat{f}_q$:

$$f_q - \frac{m}{k} \leq \hat{f}_q$$

Choose $m = 3,000,000$, $k = 12$, and $f_q = 1/4 \times 3,000,000 = 750,000$ to obtain that $\hat{f}_q \geq 500,000$. The task now reduces to finding all elements q of the stream whose estimated frequency (corresponding to the count entries from [Table 1](#)) is greater or equal to 500,000. The only element that satisfies this constraint on its estimated frequency is item: “a”.

Similarly, for [Table 2](#), choose $m = 4,000,000$, $k = 12$, and $f_q = 1/4 \times 4,000,000 = 1,000,000$ to obtain that $\hat{f}_q \geq 666,666.\bar{6}$. From [Table 2](#), the only elements satisfying this constraint are items “a” and “b”.

— Elements that “must” occur at least 25% of the time

Elements of S1 that are guaranteed to occur at least 25% times must satisfy the following constraint:

$$\hat{f}_q \geq f_q = 1/4 \times 3,000,000 = 750,000$$

From [Table 1](#), the only item that exceeds such frequency is item “a”.

Similarly, the elements of S2 that must occur at least 25% of the time must satisfy the following constraint:

$$\hat{f}_q \geq f_q = 1/4 \times 4,000,000 = 1,000,000$$

From [Table 2](#), the only item that exceeds such frequency is item “a”.

- (B) Below is the output of running a fully fledged vanilla implementation of Count Min Sketch on S1.txt:

label	"a"	"b"	"c"
count	899,566	659,805	360,551
ratio	0.29986	0.21993	0.12018

Table 3. Output of Count Min Sketch algorithm on stream S1. See [Listing 2](#) for a fuller explanation on how these frequencies have been generated.

Below is the output of running a fully fledged vanilla implementation of Count Min Sketch on S2.txt:

label	"a"	"b"	"c"
count	2,000,100	839,401	400,625
ratio	0.50003	0.20985	0.10016

Table 4. Output of Count Min Sketch algorithm on stream S2. See Listing 2 for a fuller explanation on how these frequencies have been generated.

— **Elements that “might” occur at least 25% of the time**

From Section 11.2.3 in M4D, irrespective of the randomness incurred by the hash functions, the following invariance overestimate always holds:

$$\hat{f}_q \geq f_q$$

Therefore, for stream S1, we want to enforce $\hat{f}_q \geq 1/4 \times 3,000,000 = 750,000$ for items that “might” occur at least 25% of the time. From Table 3, the only item satisfying this constraint is item “a”.

Likewise, for stream S2 we want to enforce $\hat{f}_q \geq 1/4 \times 4,000,000 = 1,000,000$ for items that “might” occur at least 25% of the time. From Table 4, the only item satisfying this constraint is item “a”.

— **Elements that “must” occur at least 25% of the time**

Elements of S1 that are guaranteed to occur at least 25% of the time must satisfy the following constraint:

$$\hat{f}_q > f_q + \epsilon F_1$$

Choose $\epsilon = 2/k = 1/6$ to obtain for stream S1 that $\hat{f}_q > 1,250,000$. Inspecting the values from Table 3, item “a” is the only item satisfying this constraint.

Analogously, for the same fixed choice of $\epsilon = 1/6$, in stream S2 we obtain $\hat{f}_q > 1,666,666.\bar{6}$. Item “a” is the only item satisfying this constraint with an estimated frequency of 2,000,100.

- (C) Misra Gries – use a hash map with keys representing labels/words and values representing their corresponding indices in the counter.

Count Min Sketch – compute each hashed word by summing up the ascii values of its constituent characters and then use modular hashing. To circumvent the problem of having anagrams being hashed to the same location, multiply the ascii values of the characters by the position their appear in the word.

- (D) While the space for the Count-Min Sketch is larger than the Misra-Gries Sketch, it has an advantage that its guarantees hold under the so-called turnstile streaming model. Under this model, each element $a_i \in A$ can either add one or subtract one from corpus, but each count f_j must remain non-negative. Thus Count-Min has the same guarantees in the turnstile model, but Misra-Gries does not.

Appendix

```
1 import numpy as np
2
3 from numba import njit
4
5 K = 12
6 DATASET_ONE = 'data/S1.txt'
7 DATASET_TWO = 'data/S2.txt'
8
9 def read_file(filename: str) -> np.ndarray:
10     with open(filename) as file:
11         corpus = file.readline()
12         corpus_ = np.array(list(map(ord, corpus)))
13         return corpus_
14
15 @njit('int64(int64[:,], int64)')
16 def find_item(label: np.ndarray, item: int) -> int:
17     for i in range(label.shape[0]):
18         if label[i] == item:
19             return i
20     return -1
21
22 @njit('UniTuple(int64[:,], 2)(int64[:,], int64)')
23 def misra_gries(S: np.ndarray, K: int) -> tuple:
24     count = np.zeros(K-1, dtype=np.int64)
25     label = np.zeros(K-1, dtype=np.int64)
26
27     for i in range(S.shape[0]):
28         j = find_item(label, S[i])
29         l = find_item(count, 0)
30         if j != -1:
31             count[j] += 1
32         elif l != -1:
33             label[l] = S[i]
34             count[l] = 1
35         else:
36             count -= 1
37
38     return count, label
39
40 def main() -> int:
41     all_files = list(map(read_file, [DATASET_ONE, DATASET_TWO]))
42     for S in all_files:
43         count, label = misra_gries(S, K)
44         for i in range(K-1):
45             print(f'{chr(label[i])} | {count[i]:{7}} | {count[i] / S.shape[0]:.5f}')
46         print()
47     return 0
48
49 if __name__ == '__main__':
50     raise SystemExit(main())
```

Listing 1. fast-misra-gries.py

```

1  import numpy as np
2
3  from numba import njit
4
5  K, T = 12, 6
6  DATASET_ONE = 'data/S1.txt'
7  DATASET_TWO = 'data/S2.txt'
8
9  np.random.seed(42)
10
11 def read_file(filename: str) → np.ndarray:
12     with open(filename) as file:
13         corpus = file.readline()
14         corpus_ = np.array(list(map(ord, corpus)))
15     return corpus_
16
17 def rand_salts() → np.ndarray:
18     return np.random.randint(T, size=T)
19
20 @njit('int64(int64, int64)')
21 def hash_func(salt: int, item: int) → int:
22     return hash((salt, item)) % K
23
24 @njit('int64[:,:](int64[:,], int64, int64, int64[:,])')
25 def count_min(S: np.ndarray, K: int, T: int, salts: np.ndarray) → np.ndarray:
26     count = np.zeros((T,K), dtype=np.int64)
27     for i in range(S.shape[0]):
28         for j in range(T):
29             l = hash_func(salts[j], S[i])
30             count[j,l] += 1
31     return count
32
33 @njit('int64(int64[:,:], int8, int64[:,])')
34 def query_min(count: np.ndarray, char: int, salts: np.ndarray) → int:
35     char_count = np.zeros(count.shape[0], dtype=np.int64)
36     for j in range(count.shape[0]):
37         l = hash_func(salts[j], char)
38         char_count[j] = count[j,l]
39     return char_count.min()
40
41 def main() → int:
42     all_files = list(map(read_file, [DATASET_ONE, DATASET_TWO]))
43     rnd_salts = rand_salts()
44     for S in all_files:
45         count = count_min(S, K, T, rnd_salts)
46         query_vals = ['a', 'b', 'c']
47         count_mins = list(map(lambda char: query_min(count, ord(char), rnd_salts), query_vals))
48         for char, char_count in zip(query_vals, count_mins):
49             print(f'{char} | {char_count:7} | {char_count / S.shape[0]:.5f}')
50     return 0
51
52 if __name__ == '__main__':
53     raise SystemExit(main())

```

Listing 2. count-min-sketch.py